SE1020 – Object Oriented Programming

Lab Sheet 05 – Solutions

Name: Vishmina Sakith Gurusinghe | **Student ID:** IT22923042 | **Faculty of Computing, SLIIT**

Question 1 – Pizza Restaurant System (Java)

Design summary

The `PizzaOrder` class stores the order ID, customer name, pizza size, topping count and delivery choice. Pricing rules follow the lab sheet.

Size	Base price
Small	$8.00
Medium	$10.00
Large	$12.00

- Each extra topping adds **$1.50**.
- Home delivery adds **$5.00**.
- Estimated preparation time = **15 minutes + (2 minutes × number of toppings)**.

Part	Requirement	Implemented as
(a)	Class with 5 attributes	`PizzaOrder` with `orderID`, `customerName`, `size`, `toppingCount`, `isDelivery`
(b)	Default constructor	`PizzaOrder()` sets default values
(c)	Overloaded constructors	`PizzaOrder(orderID, customerName)` and `PizzaOrder(orderID, customerName, size, toppingCount, isDelivery)`
(d)	Getter and setter for size with validation	`getSize()` and `setSize()`; an invalid size becomes "Small"

Part	Requirement	Implemented as
(e)	Base price by size	`getBasePrice()`
(f)	Method overloading	`calculateTotalPrice()` and `calculateTotalPrice(double discountPercentage)`
(g)	Preparation time	`estimatePreparationTime()`
(h)	Display order	`displayOrderDetails()`

PizzaOrder.java

```
public class PizzaOrder {

    // Constants
    private static final double SMALL_PRICE = 8.00;
    private static final double MEDIUM_PRICE = 10.00;
    private static final double LARGE_PRICE = 12.00;
    private static final double TOPPING_PRICE = 1.50;
    private static final double DELIVERY_FEE = 5.00;
    private static final int BASE_PREP_TIME = 15;
    private static final int PREP_TIME_PER_TOPPING = 2;

    // Attributes
    private String orderID;
    private String customerName;
    private String size;
    private int toppingCount;
    private boolean isDelivery;

    // (b) Default constructor
    public PizzaOrder() {
        this.orderID = "Unknown";
        this.customerName = "Unknown";
        this.size = "Small";
        this.toppingCount = 0;
        this.isDelivery = false;
    }

    // (c)(i) orderID and customerName only
    public PizzaOrder(String orderID, String customerName) {
```

```
        this();
        this.orderID = orderID;
        this.customerName = customerName;
    }

    // (c)(ii) all five attributes
    public PizzaOrder(String orderID, String customerName, String size,
                      int toppingCount, boolean isDelivery) {
        this.orderID = orderID;
        this.customerName = customerName;
        setSize(size);
        setToppingCount(toppingCount);
        this.isDelivery = isDelivery;
    }

    // (d) Getter and setter for size
    public String getSize() {
        return size;
    }

    public void setSize(String size) {
        if ("Small".equals(size) || "Medium".equals(size) ||
"Large".equals(size)) {
            this.size = size;
        } else {
            this.size = "Small";
        }
    }

    // Other getters and setters
    public String getOrderID() { return orderID; }
    public void setOrderID(String orderID) { this.orderID = orderID; }

    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) { this.customerName =
customerName; }

    public int getToppingCount() { return toppingCount; }
    public void setToppingCount(int toppingCount) {
        this.toppingCount = (toppingCount < 0) ? 0 : toppingCount;
    }

    public boolean isDelivery() { return isDelivery; }
```

```
    public void setDelivery(boolean isDelivery) { this.isDelivery =
isDelivery; }

    // (e) Base price according to size
    public double getBasePrice() {
        switch (size) {
            case "Medium":
                return MEDIUM_PRICE;
            case "Large":
                return LARGE_PRICE;
            default:
                return SMALL_PRICE;
        }
    }

    // (f)(i) Total price using stored values
    public double calculateTotalPrice() {
        double total = getBasePrice() + (toppingCount * TOPPING_PRICE);
        if (isDelivery) {
            total += DELIVERY_FEE;
        }
        return total;
    }

    // (f)(ii) Total price after discount (overloaded)
    public double calculateTotalPrice(double discountPercentage) {
        double total = calculateTotalPrice();
        if (discountPercentage < 0 || discountPercentage > 100) {
            System.out.println("Invalid discount percentage. No discount
applied.");
            return total;
        }
        return total - (total * discountPercentage / 100.0);
    }

    // (g) Preparation time estimator
    public int estimatePreparationTime() {
        return BASE_PREP_TIME + (PREP_TIME_PER_TOPPING * toppingCount);
    }

    // (h) Display full order details
    public void displayOrderDetails() {
        System.out.println("-------------- Order Details --------------");
```

```
        System.out.println("Order ID           : " + orderID);
        System.out.println("Customer Name      : " + customerName);
        System.out.println("Pizza Size         : " + size);
        System.out.println("Base Price         : $" + String.format("%.2f",
getBasePrice()));
        System.out.println("Number of Toppings : " + toppingCount);
        System.out.println("Home Delivery      : " + (isDelivery ? "Yes" :
"No"));
        System.out.println("Total Price        : $" + String.format("%.2f",
calculateTotalPrice()));
        System.out.println("Preparation Time   : " +
estimatePreparationTime() + " minutes");
        System.out.println("-----------------------------------------");
        System.out.println();
    }
}
```

Main.java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // First order
        PizzaOrder order1 = new PizzaOrder("O3001", "Olivia Green", "Medium",
3, true);
        order1.displayOrderDetails();

        // Second order with discount
        PizzaOrder order2 = new PizzaOrder("O3002", "Noah Smith", "Large", 2,
false);
        order2.displayOrderDetails();
        System.out.println("Enter the discount percentage: ");
        double percentage = sc.nextDouble();
        System.out.println("Order 02 - Price after the discount: $"
                + String.format("%.2f",
order2.calculateTotalPrice(percentage)));
        System.out.println();

        // Third order using default constructor
```

```
        PizzaOrder order3 = new PizzaOrder();
        order3.displayOrderDetails();

        sc.close();
    }
}
```

Sample output (discount entered: 10)

```
-------------- Order Details --------------
Order ID           : O3001
Customer Name      : Olivia Green
Pizza Size         : Medium
Base Price         : $10.00
Number of Toppings : 3
Home Delivery      : Yes
Total Price        : $19.50
Preparation Time   : 21 minutes
-----------------------------------------

-------------- Order Details --------------
Order ID           : O3002
Customer Name      : Noah Smith
Pizza Size         : Large
Base Price         : $12.00
Number of Toppings : 2
Home Delivery      : No
Total Price        : $15.00
Preparation Time   : 19 minutes
-----------------------------------------

Enter the discount percentage:
Order 02 - Price after the discount: $13.50

-------------- Order Details --------------
Order ID           : Unknown
Customer Name      : Unknown
Pizza Size         : Small
Base Price         : $8.00
Number of Toppings : 0
```

```
Home Delivery      : No
Total Price        : $8.00
Preparation Time   : 15 minutes
-----------------------------------------
```

Verification: Order 1 = $10.00 + (3 × $1.50) + $5.00 = $19.50, with 15 + (2 × 3) = 21 minutes. Order 2 = $12.00 + (2 × $1.50) = $15.00, and a 10% discount gives $13.50.

Question 2 – VTV Cable TV: Noun-Verb Analysis

(a) Nouns that can be potential classes

VTV, Cable TV company, entertainment, TV channels (Channel), customer, person, registration, customer details, package, customer number, Customer table, Subscription table, package details, bill (detailed bill, monthly bill), month, data, Channel details table, website, online tasks, subscription, list of channels, weekly TV schedule (TV schedule), email, credit card, credit card details, amount, system, Payments table, payment details, payment.

(b) Elimination using the five rules

The five rules used are: **R1 – Redundant** (same concept under another name), **R2 – Adjective** (a describing word, not a thing), **R3 – Attribute** (a property of another class), **R4 – Irrelevant** (outside the scope of the system), and **R5 – Vague** (not clearly defined, or an event or operation).

Noun	Decision	Rule	Reason
VTV / Cable TV company	Rejected	R4	The organisation itself is outside the system being modelled
Entertainment	Rejected	R2	Describes the channels; not a thing
TV channels	**Accepted**	–	Becomes class **Channel**
Customer	**Accepted**	–	Becomes class **Customer**
Person	Rejected	R1	Same as Customer
Registration	Rejected	R5	An event or operation of Customer

Noun	Decision	Rule	Reason
Customer details	Rejected	R3	Attributes of Customer (name, address, phone)
Package	Rejected	R1	The channels a customer selects are held by Subscription
Customer number	Rejected	R3	Attribute of Customer
Customer table	Rejected	R1	Storage for Customer
Subscription table	Rejected	R1	Storage for Subscription
Package details	Rejected	R3	Attribute of Subscription
Bill (detailed / monthly bill)	**Accepted**	–	Becomes class **Bill**; "detailed" and "monthly" are adjectives (R2)
Month	Rejected	R3	Billing month is an attribute of Bill
Data	Rejected	R5	Too general to be a class
Channel details table	Rejected	R1	Storage for Channel
Website	Rejected	R4	User interface, not a domain class
Online tasks	Rejected	R5	Operations, not a thing
Subscription	**Accepted**	–	Becomes class **Subscription**
List of channels	Rejected	R1	Collection of Channel held by Subscription
Weekly TV schedule	**Accepted**	–	Becomes class **TVSchedule**; "weekly" is an adjective (R2)
Email	Rejected	R3	Email address is an attribute of Customer
Credit card	**Accepted**	–	Becomes class **CreditCard**
Credit card details	Rejected	R3	Attributes of CreditCard
Amount	Rejected	R3	Attribute of Payment
System	Rejected	R5	Too vague; refers to the whole application
Payments table	Rejected	R1	Storage for Payment
Payment details	Rejected	R1	Same as Payment

Noun	Decision	Rule	Reason
Payment	**Accepted**	–	Becomes class **Payment**

Final set of classes: Customer, Channel, Subscription, Bill, Payment, CreditCard, TVSchedule.

(c) CRC cards (5 classes)

Card 1

Class: **Customer**	
Responsibilities	**Collaborators**
Register with VTV and receive a unique customer number	Subscription
Store and update customer details (name, address, email)	Subscription
Add channels to the subscription	Subscription, Channel
Remove channels from the subscription	Subscription, Channel
Provide the list of channels for the weekly TV schedule	Subscription, TVSchedule
Pay the monthly bill online	Payment, Bill

Card 2

Class: **Subscription**	
Responsibilities	**Collaborators**
Store the package (channels) selected by a customer	Customer, Channel
Add a channel to the package	Channel
Remove a channel from the package	Channel

Class: **Subscription**	
Store channels selected for the weekly TV schedule	TVSchedule
Provide subscribed channels for bill generation	Bill

Card 3

Class: **Channel**	
Responsibilities	**Collaborators**
Store channel details (channel ID, name, monthly charge)	–
Provide the channel charge for billing	Bill
Store and provide the channel's TV schedule	TVSchedule

Card 4

Class: **Bill**	
Responsibilities	**Collaborators**
Generate the monthly detailed bill	Customer, Subscription, Channel
Calculate the total amount payable	Channel
Send the bill to the customer every month	Customer
Keep the paid or unpaid status	Payment

Card 5

Class: **Payment**	
Responsibilities	**Collaborators**
Accept the credit card details and amount	Customer, CreditCard
Validate the payment	CreditCard

Class: **Payment**	
Record the payment details	Bill
Update the bill status after payment	Bill

Question 3 – Online Bookshop: Noun-Verb Analysis

(a) Nouns that can be potential classes

Online bookshop, things, books (Book), magazines (Magazine), CDs (CD), products (Product), information, price, title, specific information, author, number of pages, artists, type, registered customers (Customer), orders (Order), shipments (Shipment), store, personal customers, business customers, payments (Payment), delivery, invoice (Invoice), credit card (CreditCard), contact information, personal information, authentication, invoicing, customer types, bookshop system, reports (Report).

(b) Elimination using the five rules

Noun	Decision	Rule	Reason
Online bookshop / Bookshop system	Rejected	R4	The whole system or organisation, not a domain class
Things	Rejected	R1	Same as Product
Books	**Accepted**	–	Becomes class **Book**, a subclass of Product
Magazines	**Accepted**	–	Becomes class **Magazine**, a subclass of Product
CDs	**Accepted**	–	Becomes class **CD**, a subclass of Product
Products	**Accepted**	–	Becomes superclass **Product** (common price and title)
Information	Rejected	R5	Too general
Price, title	Rejected	R3	Attributes of Product
Specific information	Rejected	R5	Vague; becomes subclass attributes

Noun	Decision	Rule	Reason
Author, number of pages	Rejected	R3	Attributes of Book
Artists	Rejected	R3	Attribute of CD
Type	Rejected	R3	Attribute of Magazine or CD
Registered	Rejected	R2	Adjective describing customers
Customers	**Accepted**	–	Becomes superclass **Customer**
Orders	**Accepted**	–	Becomes class **Order** (may contain several products)
Shipments	**Accepted**	–	Becomes class **Shipment** (one order can have several shipments)
Store	Rejected	R1	Same as the bookshop (R4)
Personal customers	**Accepted**	–	Becomes class **PersonalCustomer**, a subclass of Customer (pays at delivery)
Business customers	**Accepted**	–	Becomes class **BusinessCustomer**, a subclass of Customer (receives an invoice)
Payments	**Accepted**	–	Becomes class **Payment**
Delivery	Rejected	R1	Same as Shipment; also an event
Invoice	**Accepted**	–	Becomes class **Invoice** (sent to business customers)
Credit card	**Accepted**	–	Becomes class **CreditCard** (payment method)
Contact information, personal information	Rejected	R3	Attributes of Customer
Authentication	Rejected	R5	An operation, not a thing
Invoicing	Rejected	R5	An operation; the thing is Invoice
Customer types	Rejected	R1	Covered by PersonalCustomer and BusinessCustomer

Noun	Decision	Rule	Reason
Reports	**Accepted**	–	Becomes class **Report** (product reports)

Final set of classes: Product, Book, Magazine, CD, Customer, PersonalCustomer, BusinessCustomer, Order, Shipment, Payment, Invoice, CreditCard, Report.

Inheritance: Book, Magazine and CD inherit from Product. PersonalCustomer and BusinessCustomer inherit from Customer.

Git submission (Question 3)

Commit this answer with a meaningful message:

```
git add .
git commit -m "Lab05 - Question03"
git push
```